

Reviewer Pack — Minimal Rank of Brauer Groups over Frege Proof Width

Entry e8ccba46fad6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 09:12:30 UTC

Minimal Rank of Brauer Groups over Frege Proof Width

Entry ID: e8ccba46fad6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 09:12:30 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Brauer Groups) **Field B** (complexity object): Complexity Theory: Frege Proof Complexity

Statement:

[‘For every Boolean formula F with n variables, the minimal rank of the Brauer group $B(F)$ is polynomially related to the Frege proof width of F .’, ‘More precisely, there exists a constant c such that for all Boolean formulas F , $|B(F)| \leq c \cdot \text{FregeProofWidth}(F)$.’, ‘Furthermore, for every set S of clauses with n variables, if S is unsatisfiable and has Frege proof width less than some threshold K , then $|B(S)| = 0$.’]

Rationale (proposer’s reasoning):

[‘Brauer groups are a fundamental tool in representation theory and algebraic topology, which have seen limited application in complexity theory. Their structure might reveal new insights into the intrinsic difficulty of proving or refuting satisfiability.’, ‘Frege proof width is a measure of the depth of a formula in the Frege hierarchy, which has been used to study circuit complexity. This conjecture could suggest that certain representation-theoretic properties are intrinsic to computational hardness.’]

Taxonomy category: Representation Theory - Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): acdef343e8d73b53

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if all 30 seeds yield a polynomial relationship between the minimal rank of the Brauer group and the Frege proof width for Boolean formulas with n variables, where the ratio $|B(F)| / \text{FregeProofWidth}(F) \leq c$ for some constant c . The conjecture is falsified if any seed produces a ratio $> c$ or a counterexample where $|B(S)| \neq 0$ for an unsatisfiable set S with Frege proof width $< K$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Brauer Groups' AND 'Frege Proof Width' AND polynomial relationship - 'minimal rank of Brauer group' AND 'Frege proof complexity' AND 'Boolean formula' - unsatisfiable clauses 'Frege proof width' AND 'Brauer group rank'

Top relevant hits considered: - [http://arxiv.org/abs/1007.3011v1] Counting lifts of Brauer characters - [http://arxiv.org/abs/2308.14302v2] Finite simple characteristic quotients of the free group of rank 2 - [http://arxiv.org/abs/1110.2956v2] Brauer spaces for commutative rings and structured ring spectra - [http://arxiv.org/abs/1203.3706v2] On the Complexity of Computing Minimal Unsatisfiable LTL formulas - [http://arxiv.org/abs/1505.02318v2] Parameters for minimal unsatisfiability: Smarandache primitive numbers and full clauses - [http://arxiv.org/abs/1604.01288v1] Unsatisfiable hitting clause-sets with three more clauses than variables

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_formula(n):
        if n == 1:
            return random.choice(['x', '¬x'])
        else:
            subformulas = [generate_boolean_formula(random.randint(1, n-1)) for _ in range(2)]
            operator = random.choice(['&', '|', '^'])
            return f'({subformulas[0]} {operator} {subformulas[1]})'

    def frege_proof_width(formula):
        if formula.startswith('(') and formula.endswith(')'):
            formula = formula[1:-1]
        if '&' in formula or '|' in formula:
            return max(frege_proof_width(subformula) for subformula in formula.split('&') + formula.split('|'))
        elif '^' in formula:
            return frege_proof_width(formula.split('^')[0]) + 1
        else:
            return 1
```

```

        return 1

def brauer_group_rank(formula):
    # Placeholder function to simulate Brauer group rank calculation
    # This is a dummy implementation and should be replaced with actual logic
    return len(formula)

n = random.randint(5, 30)
formula = generate_boolean_formula(n)
width = frege_proof_width(formula)
rank = brauer_group_rank(formula)

if rank > 10 * width: # Placeholder condition to simulate polynomial relationship
    return {
        "metric_name": "Brauer group rank",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"Formula {formula} with width {width} and rank {rank}"
    }

    return {
        "metric_name": "Brauer group rank",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
        support_fraction = len([result for result in results if result["conjecture_holds"]]) / len(results)
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}\"")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
.split(' | '))
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0bf5902f.py", line 33, in <genexpr>
    return max(frege_proof_width(subformula) for subformula in formula.split(' & ') + formula.split('
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0bf5902f.py", line 33, in frege_proof_w
    return max(frege_proof_width(subformula) for subformula in formula.split(' & ') + formula.split('
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0bf5902f.py", line 33, in <genexpr>
    return max(frege_proof_width(subformula) for subformula in formula.split(' & ') + formula.split('
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0bf5902f.py", line 33, in frege_proof_w
    return max(frege_proof_width(subformula) for subformula in formula.split(' & ') + formula.split('
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0bf5902f.py", line 33, in <genexpr>
    return max(frege_proof_width(subformula) for subformula in formula.split(' & ') + formula.split('
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0bf5902f.py", line 33, in frege_proof_w
    return max(frege_proof_width(subformula) for subformula in formula.split(' & ') + formula.split('
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0bf5902f.py", line 33, in <genexpr>
    return max(frege_proof_width(subformula) for subformula in form
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the polynomial relationship between the minimal rank of the Brauer group and the Frege proof width as required by the conjecture. | next: Re-run the test with proper error handling to ensure it completes without crashing. If the test passes, re-evaluate the results against the support condition.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12026
2	preregistration	ollama_remote	glm4:latest	0	0	6520
3	novelty	ollama_remote	glm4:latest	0	0	4872
4	novelty	ollama_remote	glm4:latest	0	0	5829
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	28791
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8504
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6769
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8809
9	judge	ollama_remote	glm4:latest	0	0	15047

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 97166 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/e8ccba46fad6.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/e8ccba46fad6.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/e8ccba46fad6.tar.gz (if generated)